Chapter 4: Templates

1. Overview

An Angular template is HTML augmented with Angular-specific syntax that tells the compiler how to bind data, listen to events, instantiate other views, and structure the DOM dynamically. Templates are not evaluated at runtime like a templating engine (e.g., Handlebars, Mustache); they are **compiled ahead of time (AOT) into JavaScript instruction sets** by the Ivy compiler. The result is a `ɵɵtemplate` / `ɵɵelementStart` / `ɵɵadvance`-style render function that the change detector re-runs on every tick.

Understanding templates deeply means understanding three separate but related layers:

1. **Syntax layer** — interpolation, property/event/attribute/class/style bindings, template expressions vs. statements, template reference variables, `ng-template`, `ng-container`, structural directives (`*ngIf`, `*ngFor`, `*ngSwitch`) and the modern built-in control flow (`@if`, `@for`, `@switch`) introduced in Angular 17.
2. **Compilation layer** — how the template parser turns markup into an AST, and how the Ivy compiler emits instructions that build/update the DOM, including the notion of "template context" and embedded views.
3. **Type-checking layer** — how the Angular compiler (via `ngtsc`) type-checks bindings against your component class using TypeScript, and the three strictness modes (`fullTemplateTypeCheck`, `strictTemplates`, basic).

Interviewers use this chapter's material to separate people who "have used Angular" from people who understand *why* Angular renders the way it does — e.g., why `*ngIf` creates and destroys components (losing local state) rather than hiding them, why `@for` requires `track`, and why two structural directives can't coexist on one element.

---

2. Core Concepts

2.1 Interpolation

Interpolation `{{ }}` binds a template expression's string value into text content or an attribute-like position. Angular calls `toString()` on the result (with special-casing for `null` / `undefined`, which render as empty string).

```
<p>Hello, {{ user.name }}!</p>
<img [src]="'assets/' + user.avatar" />
```

Interpolation is syntactic sugar for a property binding on `textContent` /an interpolated attribute; internally Ivy compiles `{{ expr }}` inside text nodes to `ɵɵtextInterpolate` (or `ɵɵtextInterpolate1..8` for multi-part strings, an optimization to avoid string concatenation allocations for the common case of ≤8 expressions).

2.2 Template Expressions vs. Template Statements

This is one of the most commonly *mis-explained* concepts, so precision matters.

Template expressions appear in property bindings and interpolation (`[property]="expr"`, `{{expr}}`). They are a restricted subset of JavaScript:

- No assignments (`=`, `+=`), no `new`, no chained expressions with `;` or `,`
- No increment/decrement (`++`, `--`)
- No bitwise `|` or `&` (because `|` is reserved for pipes and `&&`/`||` unaffected)
- Cannot reference global namespace (`window`, `document`, `console`, `Math` are not implicitly available — you must expose them via the component)
- Must be side-effect-free (idempotent) because they may run many times per change-detection cycle
- Support the optional chaining `?.` and nullish coalescing `??` operators (added when Angular templates began compiling more of TS syntax)
- Support the pipe operator `|`

Template statements appear in event bindings (`(click)="expr"`). They ARE allowed to have side effects because that's their entire purpose:

- Assignments are allowed: `(click)="count = count + 1"`
- Chained statements with `;` are allowed: `(click)="save(); close()"`
- They have access to the special `$event` object
- They do **not** support pipes

```
<!-- Expression: read-only, no side effects -->
<div [class.active]="isActive && !isDisabled"></div>

<!-- Statement: performs an action, allowed side effects -->
<button (click)="counter = counter + 1; log('clicked')">Click</button>
```

Interview-grade nuance: both are parsed by Angular's own expression parser (`@angular/compiler` `Parser`/`Lexer`), **not** by the TypeScript compiler directly at parse time — though `strictTemplates` later type-checks the resulting AST against your TS types.

2.3 Property, Attribute, Class, and Style Bindings

Syntax	Binds to	Example
`[property]="expr"`	DOM property	`[disabled]="isDisabled"`
`[attr.name]="expr"`	HTML attribute	`[attr.aria-label]="label"`
`[class.foo]="expr"`	single CSS class	`[class.active]="isActive"`
`[class]="expr"`	class list (string/array/object)	`[class]="{active: isActive}"`
`[style.prop]="expr"`	single style	`[style.width.px]="width"`
`(event)="stmt"`	event listener	`(click)="onClick($event)"`
`[(ngModel)]="prop"`	banana-in-a-box (two-way)	combines property + event

Key distinction: **property binding vs. attribute binding**. Most HTML "attributes" you bind (`src`, `disabled`, `value`, `href`) are actually reflected DOM **properties**, and `[prop]` binds to the property directly, bypassing the DOM's initial-attribute string coercion. You need `[attr.x]` only for attributes with no corresponding DOM property (custom `data-*` attributes, ARIA attributes, `colspan` on `<td>`, SVG attributes like `viewBox`).

```
<!-- WRONG intuition: this sets the 'disabled' ATTRIBUTE string -->
<button disabled="{{ isDisabled }}">X</button> <!-- always disabled! any string is truthy --
>

<!-- RIGHT: binds the DOM property, which is a real boolean -->
<button [disabled]="isDisabled">X</button>
```

2.4 Template Reference Variables

A `#ref` (or `ref="ref"`) declares a variable scoped to the template (and its children) that refers to:

- The DOM element it's placed on (an `HTMLElement`), or
- The component/directive instance if placed on an element hosting a component or a directive that exports itself (`exportAs`), or
- The `TemplateRef` if placed on `<ng-template>`, or
- An `NgForm` / `NgModel` instance in forms.

```
<input #box (input)="0" />
<p>You typed: {{ box.value }}</p>

<my-datepicker #dp="myDatepicker"></my-datepicker>
<button (click)="dp.open()">Open</button>
```

Rules to remember: they are visible in the same template (including sibling elements) but **not** inside `*ngIf` / `*ngFor`'s own microsyntax expression that appears *before* the variable is declared in source order in some edge cases, and they are not visible in parent components — templates are their own scope. Also, `#ref` on a plain element defaults to the native `Element`, not a wrapped/Angular object.

2.5 Template Context (`ng-template`, implicit context, `let-x`)

`<ng-template>` defines a chunk of markup that is **not rendered** by default — it's compiled to a `TemplateRef` that other code (a structural directive, `ViewContainerRef.createEmbeddedView`, `NgTemplateOutlet`) can instantiate on demand, potentially multiple times, each instantiation being an **embedded view**.

Structural directives pass a **context object** to the template when they instantiate it. By convention this object usually has an `$implicit` key (bound with plain `let-x`) plus named keys (bound with `let-x="key"`).

```
<ng-template #greeting let-name let-idx="index">
  Hello {{ name }} (#{{ idx }})
</ng-template>

<ng-container *ngTemplateOutlet="greeting; context: { $implicit: 'Sam', index: 1 }">
</ng-container>
```

`*ngFor` supplies a context object with `$implicit` (the item), plus `index`, `first`, `last`, `even`, `odd`, `count`. That's why `let item, let i = index` works — `i` binds to the `index` key of the context, and `item` (no `= key`) binds to `$implicit`.

2.6 `ng-container`

`<ng-container>` is a **logical grouping element that renders no DOM node** (it becomes an Angular comment marker, `<!--ng-container-->`, at runtime). Two big use cases:

1. Attaching a structural directive to a group of siblings without introducing an extra wrapper `<div>` (avoids breaking CSS flex/grid layouts, avoids invalid HTML like `<tr>` needing a `<div>` wrapper for `*ngIf`).
2. Rendering multiple non-adjacent templates or hosting multiple structural directives that couldn't otherwise coexist on the same real element.

```
<table>
  <tr *ngFor="let row of rows">
    <ng-container *ngIf="row.visible">
      <td>{{ row.a }}</td>
      <td>{{ row.b }}</td>
    </ng-container>
  </tr>
</table>
```

2.7 Legacy Structural Directives: `*ngIf`, `*ngFor`, `*ngSwitch`

The `*` is **microsyntax sugar** the template parser desugars into an `<ng-template>`. `*ngIf="cond"` on `<div>` becomes:

```
<ng-template [ngIf]="cond">
  <div>...</div>
</ng-template>
```

`*ngFor="let item of items; trackBy: trackFn; index as i"` desugars into `[ngForOf]`, `[ngForTrackBy]`, and a `let-item` / `let-i="index"` template.

Key mechanics:

- **Only one structural directive per element** is allowed (`*ngIf` and `*ngFor` cannot both sit on the same tag) because the desugaring can only wrap the element in one `<ng-template>`. To combine them, nest with `<ng-container>`.
- `*ngIf` **destroys and recreates** the embedded view (and therefore the component instance behind it) when the condition flips — all local component state, form state, subscriptions tied to that instance are torn down and recreated. This differs from `[hidden]` or `[style.display]`, which keep the instance alive and simply toggle CSS visibility.
- `*ngFor` re-renders based on identity by default (object identity comparison per item) unless a `trackBy` function is supplied — without `trackBy`, changing an array reference (even with the same data) can destroy/recreate all DOM nodes, hurting performance and losing focus/animation state.

- `*ngSwitch` / `*ngSwitchCase` / `*ngSwitchDefault` work like `*ngIf` chains but ensure mutual exclusivity and a fallback.

```
<div [ngSwitch]="status">
  <p *ngSwitchCase="'loading'">Loading…</p>
  <p *ngSwitchCase="'error'">Error!</p>
  <p *ngSwitchDefault>Ready</p>
</div>
```

2.8 Modern Built-in Control Flow: `@if`, `@for`, `@switch` (Angular 17+)

Angular 17 introduced a new **block syntax** compiled directly by the template parser (not implemented as directives at all — no `NgIf` / `NgFor` class involved, no need to import `CommonModule`). This was a deliberate architectural shift, motivated by:

1. **Performance** — built-in control flow generates more efficient instructions than structural directives (no directive instantiation/DI overhead, no microsyntax parsing indirection, and `@for` uses a mandatory, compiler-checked `track` expression that enables far better DOM node reuse via a keyed diffing algorithm, similar in spirit to how React/Vue key lists).
2. **Ergonomics** — no `<ng-template>` desugaring mental model needed, native `@else` / `@else if`, native empty-state (`@empty`) for loops, and it removes the "cannot combine `*ngIf` and `*ngFor`" trap because the block syntax nests naturally.
3. **Bundle size** — templates using only `@if` / `@for` / `@switch` don't need to import `CommonModule` / `NgIf` / `NgFor`, shrinking the compiled output and enabling components to be fully standalone with zero boilerplate imports for basic control flow.
4. **Type narrowing** — `@if (user; as u)` and simple truthy checks give better type narrowing in the template type checker than `*ngIf` historically did (though `*ngIf` also improved via `NgIfContext` and "let" aliasing over time).

```
@if (isLoggedIn) {
  <p>Welcome back, {{ user.name }}!</p>
} @else if (isGuest) {
  <p>Continue as guest</p>
} @else {
  <p>Please log in</p>
}

@for (item of items; track item.id; let i = $index, isFirst = $first) {
  <li>{{ i }}: {{ item.name }}</li>
} @empty {
  <li>No items found.</li>
}

@switch (status) {
  @case ('loading') { <p>Loading…</p> }
  @case ('error') { <p>Error!</p> }
  @default { <p>Ready</p> }
}
```

`@for` implicit variables: `$index`, `$first`, `$last`, `$even`, `$odd`, `$count`. The `track` expression is **mandatory** (compile error without it) — it should be a stable identifier (`item.id`), not the whole object and not array index unless items truly have no identity, because the identity returned by `track` is what the diffing algorithm uses to decide "reuse this DOM node" vs. "destroy and create a new one."

`@if...as` binding for narrowing + reuse of an async value without repeated pipe calls:

```
@if (user$ | async; as user) {
  <p>{{ user.name }}</p>
}
```

Migration: Angular ships a schematic, `ng generate @angular/core:control-flow`, that automatically rewrites `*ngIf` / `*ngFor` / `*ngSwitch` templates to `@if` / `@for` / `@switch`. As of Angular 17/18+, the built-in flow is the recommended default for new code; `*ngIf` etc. still work and are not deprecated for removal in the near term, but Angular's own style guide and CLI schematics steer new templates toward the block syntax.

2.9 `ng-template` vs `ng-container` vs `@if` block — when compiled, what's rendered

Construct	Compiles to	Renders a DOM node?	Typical use
`<ng-template>`	`ɵɵtemplate` (TemplateRef factory), not invoked unless something instantiates it	No (nothing, until instantiated)	Deferred/conditional content, `TemplateRef` for portals/overlays, `matHeaderCellDef`-style structural APIs
`<ng-container>`	comment node marker + inline instructions for its children	Comment node only	Grouping without wrapper element, hosting a structural directive without an extra tag
`@if` / `@for` / `@switch`	dedicated Ivy instructions (`ɵɵconditional`, `ɵɵrepeaterCreate` / `ɵɵrepeater`)	Only the matched branch's content (no wrapper)	Preferred modern control flow

3. Code Examples

3.1 Expression vs. Statement Contrast

```
<!-- component.html -->
<ul>
  <li *ngFor="let todo of todos; trackBy: trackByodoId">
    <!-- expression: read-only, evaluated on every CD run -->
    <span [class.done]="todo.completed">{{ todo.title }}</span>

    <!-- statement: mutates state, only runs on click -->
    <button (click)="todo.completed = !todo.completed; save(todo)">
      Toggle
    </button>
  </li>
</ul>
```

```
// component.ts
trackByodoId(index: number, todo: Todo): number {
  return todo.id; // stable identity avoids DOM churn
}
```

3.2 `ng-template` + `TemplateRef` + `ViewContainerRef` (imperative instantiation)

```
<ng-template #warning let-msg>
  <div class="warn">{{ msg }}</div>
</ng-template>

<ng-container #anchor></ng-container>
<button (click)="show()">Show warning</button>
```

```
import { Component, ViewChild, TemplateRef, ViewContainerRef } from '@angular/core';

@Component({ selector: 'app-demo', templateUrl: './demo.html' })
export class DemoComponent {
  @ViewChild('warning') warningTpl!: TemplateRef<{ $implicit: string }>;
  @ViewChild('anchor', { read: ViewContainerRef }) anchor!: ViewContainerRef;

  show(): void {
    this.anchor.clear();
    this.anchor.createEmbeddedView(this.warningTpl, { $implicit: 'Disk almost full!' });
  }
}
```

3.3 Custom Structural Directive (how `*ngIf` is built under the hood)

```
import { Directive, Input, TemplateRef, ViewContainerRef } from '@angular/core';
```

```
@Directive({ selector: '[appUnless]', standalone: true })
export class UnlessDirective {
  private hasView = false;

  constructor(
    private templateRef: TemplateRef<unknown>,
    private viewContainer: ViewContainerRef,
  ) {}

  @Input() set appUnless(condition: boolean) {
    if (!condition && !this.hasView) {
      this.viewContainer.createEmbeddedView(this.templateRef);
      this.hasView = true;
    } else if (condition && this.hasView) {
      this.viewContainer.clear();
      this.hasView = false;
    }
  }
}
```

```
<p *appUnless="isHidden">Shown when isHidden is false</p>
```

3.4 Legacy vs Modern Side by Side

```
<!-- Legacy -->
<div *ngIf="user; else noUser">Hello {{ user.name }}</div>
<ng-template #noUser>Please log in</ng-template>

<div *ngFor="let p of products; index as i; trackBy: trackByFn">
  {{ i }}: {{ p.name }}
</div>

<!-- Modern -->
@if (user) {
  <div>Hello {{ user.name }}</div>
} @else {
  <div>Please log in</div>
}

@for (p of products; track p.id; let i = $index) {
  <div>{{ i }}: {{ p.name }}</div>
}
```

3.5 Combining Conditionals and Loops Cleanly (a pain point solved by the new syntax)

```
<!-- Legacy: needs nested ng-container, can't stack *ngIf and *ngFor on one tag -->
<ng-container *ngIf="showList">
  <li *ngFor="let item of items">{{ item }}</li>
</ng-container>

<!-- Modern: nests naturally, no extra element needed -->
@if (showList) {
  @for (item of items; track item) {
    <li>{{ item }}</li>
  }
}
```

---

4. Internal Working

4.1 Compilation Pipeline (Ivy)

1. **Parsing**: The template string is tokenized/lexed and parsed into an HTML AST by `@angular/compiler`'s `Parser` and `Lexer`, producing nodes like `Element`, `Text`, `BoundText` (interpolation), `Template` (structural), `Content`.
2. **Binding parsing**: Attribute strings like `[prop]="expr"`, `(event)="stmt"`, and microsyntax (`*ngFor="let x of xs"`) are parsed into `ParsedProperty`, `ParsedEvent`, and expanded into an `ng-template` sub-AST for structural directives.
3. **Template AST → Ivy instructions**: `TemplateDefinitionBuilder` (or the newer `t2` binder + Ivy code generators) walks the AST and emits a `template()` function per component containing calls such as:
   - `ɵɵelementStart` / `ɵɵelementEnd` — open/close a DOM element
   - `ɵɵtext` / `ɵɵtextInterpolate1` — static/interpolated text nodes
   - `ɵɵproperty` — property bindings
   - `ɵɵlistener` — event bindings
   - `ɵɵtemplate` — declares a nested view (used for structural directives, `<ng-template>`, and now also for `@if` / `@for` branches)
   - `ɵɵconditional` — the Angular 17+ instruction backing `@if` / `@switch`, selecting which template index to activate
   - `ɵɵrepeaterCreate` / `ɵɵrepeater` — the instruction set backing `@for`, implementing the LiveCollection-based keyed diff driven by `track`
4. **Two-pass execution**: Every component's `template()` function is called twice per invocation cycle — once with `RenderFlags.Create` (first time only, builds the initial DOM/instantiates bindings) and once with `RenderFlags.Update` (every change-detection run, re-evaluates bindings and patches only what changed).

```
// Conceptually what Ivy generates for <p>{{ name }}</p> — simplified
function Template(rf: RenderFlags, ctx: MyComponent) {
  if (rf & RenderFlags.Create) {
    ɵɵelementStart(0, 'p');
    ɵɵtext(1);
    ɵɵelementEnd();
  }
  if (rf & RenderFlags.Update) {
    ɵɵadvance(1);
    ɵɵtextInterpolate(ctx.name);
  }
}
```

4.2 Embedded Views and the View Hierarchy

Every `<ng-template>` (explicit, or implicit via `*directive` / `@if` / `@for`) compiles to its own nested `template()` function, distinct from — but lexically nested inside — the parent's. When a structural directive (or `@if` / `@for` machinery) decides to render it, it calls `ViewContainerRef.createEmbeddedView(templateRef, context)`, which:

- Instantiates an `LView` (Ivy's internal array-based data structure holding DOM nodes, bindings, directive instances, and context for that view)
- Inserts it into the `ViewContainerRef`'s comment-anchor location in the DOM
- Registers it in the parent's view tree, so it participates in change detection top-down, in creation order

This is why `*ngIf` / `@if` toggling means **full destroy/recreate** of that `LView` (running `ngOnDestroy` on any directives/components inside), whereas things like `[hidden]` merely flip a CSS property on an existing, never-destroyed `LView`.

4.3 `@for`'s Keyed Diffing (`LiveCollection`)

`@for`'s repeater instructions maintain a live, keyed collection abstraction. On each update:

1. It computes the new list of `track` keys.
2. It diffs old-key-order vs. new-key-order using a variant of the same "keyed diff" strategy libraries like React/Vue use for lists (LCS-ish move detection), to compute a **minimal set of moves/creates/destroys**.
3. Items whose key is unchanged only get their bindings updated, not recreated — hence why choosing a *stable* `track` key (usually `item.id`) is what makes this efficient. Using `track $index` disables most of that benefit because the key is positional, not identity-based (equivalent to no `trackBy` in old `*ngFor`, except the compiler forces you to be explicit about the tradeoff).

4.4 Template Type Checking (`ngtsc`)

Angular's compiler (`ngtsc`, TypeScript-based) type-checks templates by generating a **synthetic TypeScript file** ("type check block", TCB) per component that mirrors the template's data flow as ordinary TypeScript statements, then runs it through the real TypeScript type checker. Roughly:

```
<div *ngIf="user as u">{{ u.nam }}</div>
```

conceptually lowers to something like:

```
function _tcb(ctx: MyComponent) {
  if (ctx.user) {
    const u = ctx.user;
    u.nam; // TS2339: Property 'nam' does not exist — caught!
  }
}
```

Strictness is controlled in `tsconfig.json` under `angularCompilerOptions`:

Setting	Effect
`fullTemplateTypeCheck: true` (legacy, pre-strict)	Type-checks embedded views (inside `*ngIf` / `*ngFor`), not just top-level bindings; still uses `any` liberally
`strictTemplates: true` (recommended, implies the above and more)	Full strict mode: strict input/output binding types, strict `$event` types on DOM/custom events, strict null checks honored, strict generic inference for directives (e.g., `NgForOf<T>` correctly narrows `let item` to `T`, `AsyncPipe` narrows `Observable<T> | null` correctly), strict attribute types (`[attr.x]` must be string-coercible), disables implicit `any` for template variables
Basic (neither flag)	Only top-level bindings of the component's own template are checked loosely; structural directive contents/child component inputs get much weaker/no checking

`strictTemplates` is what enables genuinely catching bugs like passing a `string` where a child component's `@Input()` expects a `number`, or calling a method on a possibly-`null` value that hasn't been narrowed by `@if` / `*ngIf`. It's why enabling it project-wide (recommended for all new projects, and CLI-generated projects have it on by default since Angular 9's Ivy strict mode) surfaces a wave of previously-silent template bugs.

---

5. Edge Cases & Gotchas

- `[disabled]="'false'"` **truthiness trap**: binding the *string* `'false'` to a boolean DOM property is still truthy. Always bind an actual boolean expression, not stringly-typed values, and never use plain attribute syntax (`disabled="{{cond}}"`) for boolean DOM properties.
- **Two structural directives on one element**: `<div *ngIf="a" *ngFor="let x of xs">` is a **compile error** — desugaring can only wrap in one `<ng-template>`. Fix with nested `<ng-container>` (legacy) or nested `@if` / `@for` blocks (modern, no such restriction because they aren't directives).
- `*ngFor` **without** `trackBy` **+ object identity churn**: replacing an array with a new array containing "the same" data (different references) destroys and rebuilds every row — losing focus, animations, and expensive child component state. `@for` forces you to supply `track`, largely preventing this class of bug at compile time.

- **Template reference variables are not visible across templates**: a `#ref` inside an `*ngFor` / `@for` is scoped to that embedded view/iteration and is not accessible from a sibling iteration or from outside the loop.
- `ng-container` **cannot itself take non-structural attribute/class/style bindings that need a rendered element** — since it produces no DOM node, `[class]="x"` on `<ng-container>` has no visual effect (it doesn't error, but there's nothing to apply the class to). It can, however, host structural directives, template variables, and `*ngIf` / `@if` / `ngComponentOutlet` type controls, which don't need a real host element.
- `@else` **must immediately follow the closing brace of** `@if` — whitespace/comments between `}` and `@else` are fine, but another element or text node in between breaks the chain (each `@if` / `@else if` / `@else` is a distinct block, and only directly adjacent blocks in the parse chain qualify).
- `@switch` **has no fallthrough** and does not require (nor support) a `break` — it behaves like a strict multi-way `@if` chain checking `===` equality, not JS `switch`'s type-coercing comparison surprises.
- **Mixing legacy and modern control flow** in the same template is fully supported (they compile independently), but mixing styles inconsistently across a codebase hurts readability/review — most teams migrate wholesale via the schematic rather than gradually per-file.
- `strictTemplates` **false negatives with** `any`**-typed data**: if a component input or the surrounding logic types something as `any` (e.g., data from an untyped API response), the template type checker can't catch mistakes — "strict" only helps as strictly as your surrounding TypeScript types are.
- `ngModel` **two-way binding needs** `FormsModule` / `name` **attribute** inside `*ngFor` — without a unique `name` (or `[ngModelOptions]="{standalone: true}"`), Angular Forms will throw duplicate form-control-name errors when the same `ngModel` template repeats across iterations without differentiation, unless wrapped correctly or you avoid template-driven forms in loops.
- **Pipes are not allowed in template statements** (`(click)="value | somePipe"` is invalid) — only in expressions/interpolation.
- `track` **expression referencing mutable object identity vs. value**: `track item` (whole object) works only if the *same object reference* persists across re-renders; if your data layer creates new object literals every fetch (common with `HttpClient` JSON parsing), you must `track item.id`, not `track item`, or every re-fetch appears as "all new" to the differ.

---

6. Interview Questions & Answers

Q1. What is the difference between a template expression and a template statement?
A: Template expressions appear in property bindings/interpolation (`[x]="expr"`, `{{expr}}`) and must be side-effect-free, restricted JS (no assignment, no `new`, no `++` / `--`). Template statements appear in event bindings (`(event)="stmt"`) and are allowed side effects, including assignment and statement chaining with `;`, plus access to `$event`. Expressions can use pipes; statements cannot.

Q2. Why does `<div *ngIf="a" *ngFor="let x of xs">` fail to compile?
A: Both `*ngIf` and `*ngFor` desugar into wrapping the host element in an `<ng-template>`. Since an element can only be wrapped in one `<ng-template>` by the parser, two structural directives on the same element is ambiguous/unsupported. The fix is nesting: put one on the element and the other on a wrapping `<ng-container>`, or in the modern syntax, simply nest `@if { @for { ... } }` blocks, which has no such restriction since they aren't directive-based.

Interviewer intent: Checks whether the candidate understands structural directives are sugar for `ng-template`, not "just modifiers," and whether they know the practical workaround.

Q3. What does `[hidden]="cond"` do differently from `*ngIf="!cond"`?
A: `[hidden]` toggles the CSS `display` via the DOM `hidden` property/attribute while keeping the element (and any component/directives on it) fully instantiated in the DOM and in the Angular view tree — state, subscriptions, and lifecycle hooks are untouched. `*ngIf`/`@if` conditionally instantiate or fully destroy the embedded view — `ngOnDestroy` runs, all local component state is lost, and it's recreated fresh the next time the condition becomes true. `*ngIf` is heavier but frees memory/DOM nodes and skips change detection on the hidden subtree entirely; `[hidden]` is cheaper to toggle but the subtree still exists (and is still checked by CD unless `OnPush` skips it).

Q4. Why was `@for` designed to require a mandatory `track` expression, unlike optional `trackBy` in `*ngFor`?
A: In legacy `*ngFor`, omitting `trackBy` silently falls back to identity-based diffing, which frequently causes accidental full-list DOM teardown/rebuild whenever the array reference changes (e.g., after refetching from an API), a very common and hard-to-spot performance bug. By making `track` a compiler-enforced, non-optional part of the `@for` syntax, Angular forces developers to consciously choose an identity key at write time, catching the mistake at compile time instead of leaving it as a silent runtime performance trap. It also gives the new keyed-diffing (`LiveCollection`) algorithm a guaranteed identity to work with, enabling more consistent DOM-node reuse.
Interviewer intent: Tests whether the candidate can articulate the *design rationale* behind an API change, not just recite "track is required," which signals deeper framework understanding versus rote memorization.

Q5. What is a template reference variable, and where is it scoped?
A: Declared with `#name` (or `name="exportAsName"`), it holds a reference to the DOM element, the directive/component instance (via `exportAs`), or a `TemplateRef` if placed on `<ng-template>`. It's scoped to the template it's declared in (and descendants of that template/embedded view) — it is not visible to sibling templates, parent components, or across separate `*ngFor`/`@for` iterations, since each iteration gets its own embedded view/scope.

Q6. Explain what `ng-container` compiles to and why you'd use it.
A: It compiles to an HTML comment node marker at runtime (no real element in the DOM) while still participating fully in Angular's structural/binding system. It's used to (a) attach a structural directive to a group of sibling elements without introducing an extra wrapper element that could break CSS layout (flex/grid) or violate HTML nesting rules (e.g. `<tr>`/`<td>` requiring only valid table children), and (b) combine multiple templates/conditions without polluting the DOM.

Q7. What is the "context" object in `ng-template`, and how does `let-x` relate to it?
A: When a structural directive or `NgTemplateOutlet` instantiates a `TemplateRef` as an embedded view, it supplies a context object. `let-x` in the template binds local template variable `x` to the context's `$implicit` property by default, or to a named key when written `let-x="key"`. E.g., `*ngFor` supplies `{ $implicit: item, index, first, last, even, odd, count }`, which is why `let item` captures the array element and `let i = index` captures the loop index.

Q8. What's the difference between `fullTemplateTypeCheck` and `strictTemplates` in `angularCompilerOptions`?
A: `fullTemplateTypeCheck` extends type checking into nested/embedded views (inside `*ngIf`, `*ngFor`, etc.) rather than only the top-level template, but still allows a fair amount of implicit looseness. `strictTemplates` is

a superset: strict input/output binding type checks against `@Input()` / `@Output()` declarations, strict `$event` typing for both DOM and custom events, correct strict generic inference for structural directives (`NgForOf<T>`, `AsyncPipe`), and strict null handling — surfacing real bugs like passing the wrong type to a child component's input. Modern Angular CLI projects enable `strictTemplates: true` by default.

Q9. Why does Angular disallow assignment operators in template expressions (`[x]="a = b"`) but allow them in event bindings?
A: Property-binding/interpolation expressions may be evaluated multiple times per change-detection pass (Angular re-evaluates bindings to check for changes), so they must be pure/idempotent — an assignment would silently mutate state as a side effect of merely *rendering*, causing unpredictable and possibly infinite-loop-prone behavior (and would trip `ExpressionChangedAfterItHasBeenCheckedError` in dev mode). Event bindings, by contrast, run exactly once per triggered DOM event, so intentional, controlled side effects (assignments, method calls, chained statements) are both safe and the entire point.
Interviewer intent: Probes understanding of Angular's change-detection execution model, not just template syntax rules memorized in isolation.

Q10. How does the Angular compiler generate the "Create" vs "Update" passes for a component template, and why?
A: Ivy compiles each template into a single function invoked with a `RenderFlags` bitmask. On first render it's called with `RenderFlags.Create`, executing only the DOM-construction instructions (`ɵɵelementStart`, `ɵɵtext`, etc.) to build the initial DOM tree once. On every subsequent change-detection run it's called with `RenderFlags.Update`, executing only the binding-evaluation/patch instructions (`ɵɵproperty`, `ɵɵtextInterpolate`, etc.) that diff and update existing nodes, without re-creating elements. Splitting these lets Angular avoid the cost of re-walking/re-creating the DOM structure on every CD cycle — only bindings are re-evaluated.

Q11. What actually happens internally when `*ngIf`'s condition flips from `true` to `false`?
A: The `NgIf` directive (holding injected `TemplateRef` and `ViewContainerRef`) calls `viewContainerRef.clear()`, which destroys the embedded `LView` for that `<ng-template>` — running `ngOnDestroy` on any directives/components within it, removing its DOM nodes, and detaching it from the view tree so it's no longer checked during change detection. When it flips back to `true`, `createEmbeddedView` builds a brand-new `LView` (running constructors, `ngOnInit`, etc., fresh) and re-inserts it at the container's comment anchor.

Q12. What's the difference in bundle/import requirements between using `*ngIf` / `*ngFor` and `@if` / `@for` in a standalone component?
A: `*ngIf`, `*ngFor`, `*ngSwitch` are directives (`NgIf`, `NgForOf`, `NgSwitch`) that must be explicitly imported (individually or via `CommonModule`) into a standalone component's `imports` array, and they add their directive class code and directive-matching machinery to the compiled output. `@if` / `@for` / `@switch` are parsed and compiled directly into dedicated Ivy instructions (`ɵɵconditional`, `ɵɵrepeaterCreate`) with **no directive class, no import, and no DI-based directive lookup involved** — reducing both bundle size and the runtime overhead of directive matching/instantiation.

Q13. Why can't you put a template reference variable and read it in the *same* binding expression on that element (e.g., `<input #box [value]="box.value">`)?
A: Template reference variables are only fully resolved for use *after* the element they annotate has been created — practically, Angular's binding evaluation order means a `#ref` is available to sibling/descendant bindings within the same template but attempting self-referential circular reads on the very element declaring the variable in certain binding orders is unreliable/unsupported; the safe pattern is to reference `#box` from

other elements (e.g., a later `<p>{{box.value}}</p>`), or from event handlers/statements on the same element (`(input)="0"` triggers CD, then interpolation elsewhere reads `box.value`), not from a same-element competing property binding expecting simultaneous resolution.

Q14. How would you fix a performance problem where a `*ngFor`-rendered list of 5,000 rows re-renders entirely every time a WebSocket pushes new data (even though only one row changed)?
A: First, ensure the array update doesn't blindly replace all object references — ideally use immutable, per-item updates (change just the affected item's reference, keep others stable) or fully immutable with correct `id`. Second, add `trackBy` (legacy) returning a stable `id`, or migrate to `@for (... ; track item.id)`, so Angular's diffing algorithm can match unchanged items by identity and skip recreating their DOM/component instances — updating only the bindings of the row(s) whose reference actually changed. Optionally combine with `ChangeDetectionStrategy.OnPush` on the row component so even matched-but-untouched rows skip re-checking their own internal bindings unless their `@Input()` reference changes.
Interviewer intent: A practical, scenario-based question checking whether the candidate can connect `trackBy` / `track`, immutability, and `OnPush` into one coherent performance story rather than reciting each concept in isolation.

---

7. Quick Revision Cheat Sheet

- **Interpolation** `{{ }}` → stringifies an expression into text/attribute position; compiles to `ɵtextInterpolate*`.
- **Expressions** (bindings/interpolation): pure, no assignment/ `new` / `++`, support pipes, may run many times per CD cycle.
- **Statements** (events): impure OK, support assignment/ `;` chaining, access `$event`, no pipes.
- `[prop]` binds a DOM property (real type); `[attr.x]` binds an HTML attribute (always stringified) — use `attr.` only when there's no DOM property equivalent (ARIA, `data-*`, SVG, `colspan`).
- `#ref` → element / component instance (`exportAs`) / `TemplateRef` if on `<ng-template>`; scoped to its own template.
- `<ng-template>` → not rendered by default; compiles to a `TemplateRef` factory; instantiated via structural directive or `ViewContainerRef.createEmbeddedView(tpl, context)`.
- **Context object**: `let-x` → `context.$implicit`; `let-x="key"` → `context.key`. `*ngFor` context: `$implicit, index, first, last, even, odd, count`.
- `<ng-container>` → renders as a comment node; no real DOM element; used to group siblings or attach a structural directive without an extra wrapper tag.
- **Only one structural directive per element** (`*ngIf` + `*ngFor` together = compile error) — nest via `ng-container`, or use `@if` / `@for` blocks which don't have this limit.
- `*ngIf` / `@if` **false** → **true**→**false** = full destroy + recreate of the embedded view (state lost, `ngOnDestroy` / `ngOnInit` run); `[hidden]` just toggles CSS, instance stays alive.
- `*ngFor` **without** `trackBy` = identity-based diffing, easy to accidentally destroy/recreate all rows on array-reference change; `@for` makes `track` **mandatory**.
- `@if / @else if / @else`, `@for ... track ... { } @empty { }`, `@switch { @case / @default }` — Angular 17+ built-in control flow; compiles to `ɵɵconditional` / `ɵɵrepeaterCreate` + `ɵɵrepeater`;

no directive import needed, smaller/faster output; keyed diffing via `LiveCollection` uses `track` as identity.

- **Migration**: `ng generate @angular/core:control-flow` schematic auto-converts `*ngIf/*ngFor/*ngSwitch` → `@if/@for/@switch`.
- **Ivy compiles templates to two-phase `template()` functions**: `RenderFlags.Create` (build DOM once) + `RenderFlags.Update` (patch bindings every CD run).
- **Template type checking (`ngtsc`)**: generates a synthetic "type check block" TS file per component and runs the real TS checker against it. `fullTemplateTypeCheck` checks inside embedded views; `strictTemplates` adds strict input/output/`$event`/generic-inference/null checks — recommended default for new projects.
- **Common gotchas**: string `"false"` is truthy for boolean bindings; pipes not allowed in statements; `ng-container` can't visually style itself (no element); `@else` must directly follow `}`; `track` needs a stable identity key, not the whole mutable object if references churn.

Created By - Durgesh Singh